

C++/systems engineer with professional LLVM and static-analysis experience. I work on optimization legality, graph algorithms, x86-64 code generation/register allocation, and differential execution, with a focus on correctness, machine-level behavior, and reproducible validation.

Professional systems / analysis

MCST: LLVM 22/C++23 optimization; ISP RAS: industrial C#.NET static analysis.

x86-64 pipeline

SSA/CFG validation, liveness/interference, linear scan, seeded simulated annealing, SysV codegen, and differential execution.

Conservative optimization

LLVM Global-IV: APInt affine evolution, transitive effects, explicit legality, and 29 verifier/idempotence/execution regressions.

EXPERIENCE

- 2026 — present

ISP RAS — Static Analysis Engineer

 - Contribute to SharpChecker, ISP RAS's industrial static-analysis platform for C#.NET.
- July 1 — August 31, 2026

MCST — Compiler Engineering Intern

 - Implemented an LLVM LICM pass in C++; transformation legality is derived from loop structure, memory accesses, side effects, and speculative-safety constraints.
 - Built a Python harness comparing execution before/after optimization and checking explicit transformation / no-transformation cases; also implemented RPO, Dijkstra, Dinic, and Tarjan SCC in C++23.

PROJECTS

- x86-64 Compiler Backend & Register Allocation

A reference interpreter and differential execution check generated-code semantics; the allocator assignment contract is verified independently.
- LLVM Interprocedural Global IV Optimization

A conservative LLVM 22 prototype/MVP with an actual IR transform; 29 positive/negative regression cases cover verifier validity, idempotence, and before/after execution.
- PS-form Memory Dependence Analyzer

Conservative yes/no/maybe results checked by an independent exact oracle on small domains, randomized/metamorphic tests, and reproducible counterexamples.

SKILLS

- C++ / toolchain

C++23, C17, LLVM 22, CMake, Linux, clang-tidy, ASan/UBSan
- Algorithms / analysis

graph algorithms, CFG, dominance, SSA, SCC, dataflow, memory dependence
- Machine-level systems

x86-64 SysV, code generation, liveness/interference, register allocation, generated-code analysis
- Verification

exact/reference oracles, differential/metamorphic testing, isolated execution, reproducible counterexamples

EDUCATION

HSE University — Software Engineering, 2026–2030

RECOGNITION

- Prize-winner in HSE Vysshaya Proba in competitive and industrial programming.

Moscow / remote